Self-Evolving Recommender Systems: A Survey and an Evaluation Audit

Shanglin Yang
Independent Researcher
USA
kudoysl@gmail.com

Zhan Shi
Santa Clara University
Santa Clara, CA, USA
ashi2@scu.edu

Abstract

In 2026 a new kind of recommender research appeared: large language model (LLM) agents that propose changes to a recommender's configuration or source code, train the changed model, and keep what the evaluation says is better. Within ten months, about thirty such systems were reported, from open-source research harnesses to production loops at Google, Meta, Alibaba, Kuaishou, Tencent and NetEase. This survey organises them, together with search-space AutoML and the agentic recommenders that update their own memory or weights, along two axes: *what* the loop changes and *which signal* decides whether a change survives. Because a self-evolving system is only as good as the signal it selects on, we then audit how 22 configuration- and code-level recommender systems are evaluated, coding each paper's full text against nine criteria. Two of the 22 evaluate with full-catalogue ranking. Of the 15 that select candidates offline, eight do not say which data split makes the decision, one selects on its test leaderboard, and three state that the test set was used once. None reports its headline result with the spread over repeated evolution sessions, and none runs an equal-budget random search over its own edit space; the few studies that measure transfer to held-out data find that part of the selected gain, sometimes most of it, does not survive. Two systems report that their agent inflated the metric or leaked the target, caught by a human or by monitoring rather than by the acceptance rule. We distil these findings into a reporting standard and describe recforge, an open benchmark under development that implements the API side of a sealed-test, full-ranking, equal-budget protocol.

Keywords

recommender systems, LLM agents, AutoML, program evolution, evaluation, reproducibility

1 Introduction

Recommender models have long been improved by a loop that engineers run by hand: propose a change, train, compare offline, and launch the winners into an online experiment. Automating parts of this loop is an old goal. Neural architecture and embedding search for click-through-rate (CTR) models [44, 74, 100, 105] and loss search [39] automated the choice inside a space that a person had written down [106]. What changed in 2026 is that the proposer became an LLM agent that writes code. Following FunSearch [72], AlphaEvolve [60] and open implementations such as OpenEvolve [61] and ShinkaEvolve [33], agents now edit the model, loss and training code of recommenders directly. Google reported such a loop for YouTube ranking [78] and Self-EvolveRec released an open harness [30] in February 2026; within months, open harnesses such as RecHarness [43] and systems at Meta [37, 52], Alibaba [23, 53], Kuaishou [34, 89, 111], Tencent [3, 48], NetEase [98] and others.

These systems are optimisers, and an optimiser exploits whatever its objective rewards. A loop that evaluates hundreds of agent-written candidates on one validation set and keeps the maximum is the setting in which the known weaknesses of recommender evaluation—sampled metrics [32], temporal leakage [26], untuned baselines [15, 70], and the reuse of a holdout for adaptive decisions [7, 17]—are most likely to turn into reported gains. The agent also controls the code that produces its score. Two of the systems we read report that their agent did exactly this: RecEvolve's agent inflated an in-batch-negative metric by shrinking the batch size [63], and an AI co-scientist for search ranking produced "metric-winning bugs" such as silently growing the model [83].

Existing surveys do not cover this development. Surveys of LLM agents for recommendation treat the agent as the recommender, a conversational interface or a user simulator [25, 66, 102]; surveys of self-evolving agents treat the agent's own prompts, memory, tools or weights as the object that evolves, and mention recommendation at most in passing (one paragraph on self-optimised fine-tuning in [2], none in [18]); the most recent roadmap for agentic recommender systems mentions the "agent as engineer" direction in one clause [42]. Beel et al. call for autonomous recommender-systems research labs and for benchmarks to evaluate them [6]. RMD-Bench [65] and ExecML-HSTU [12] each fix one setting, but no shared benchmark yet compares evolvers under a sealed test, full-ranking evaluation and equal-budget null controls.

This survey makes four contributions.

(1) **A taxonomy** (§4) that places search-space AutoML, configuration and code evolution, evolution of generative-recommender components, and self-improving agentic recommenders on two axes: the evolved object and the selection signal.
(2) **A review of the systems** (§5) at each level, with what is open source.
(3) **An evaluation audit** (§6) of the configuration- and code-level systems on nine criteria, coded from the full text with verbatim evidence that we release.
(4) **A reporting standard and a shared protocol** (§7), partly implemented in an open benchmark under development, recforge [69], together with open problems (§8).

2 Scope and Survey Method

Definition. We call a recommender system *self-evolving* when an automated proposer changes some part of it, the changed system is evaluated, and the evaluation decides which changes are kept,

repeatedly and without a human choosing each change. Humans may approve launches; the loop must generate and select candidates by itself. This includes systems whose proposer is a search algorithm over a fixed space (background, §3), but our focus is on LLM proposers.

Inclusion. We include (i) systems that change a recommender's or ranker's configuration, features, code or components under an evaluation loop; (ii) agentic or LLM recommenders whose policy, memory, skills, prompts or weights improve from feedback they generate; and (iii) benchmarks, environments and surveys for these. We exclude LLM recommenders without a self-improvement loop, conversational recommendation in general, and general hyperparameter-optimisation libraries. Search and ranking systems outside recommendation (learning to rank [64], lexical retrieval [59]) are included as comparison points when they inform evaluation.

Search protocol. The cut-off date is 21 October 2026. We started from keyword searches over arXiv, OpenAlex and Semantic Scholar ("self-evolving recommender", "LLM agent recommender model optimisation", "agentic harness recommender", "LLM-driven architecture search CTR" and survey queries), then walked the reference lists of the 2026 systems (RecHarness, EvoRec, AgentX-Model, RecEvolve) and of the agentic-recommender roadmap backwards and their titles forwards, and finished with about thirty full-text arXiv phrasings combined with company names. Generic keyword search returned mostly unrelated "agent for *X*" papers; the citation graph of the 2026 systems was the productive source. Every arXiv entry in our bibliography was generated from the arXiv API record, and every code repository we list was checked to exist.

Audit procedure. For each configuration- or code-level system, we read the full text (arXiv HTML or PDF, including appendices) and coded nine criteria (Table 1). A criterion is coded "not stated" when the paper does not say, which is different from "no". Each coded cell is backed by a verbatim quote of at most 25 words with a section pointer; the data file and evidence will be released with recforge so that the coding can be checked and corrected. For systems whose launch decision is an online experiment, we read the offline criteria as describing the offline screen, and assess the launch evidence separately (R9 in §7).

Table 1: Audit codebook.

C1	Metric: full-catalogue ranking, sampled ranking (with N negatives), AUC/loss, or online only
C2	Selection signal: which split or signal accepts or rejects each candidate
C3	Test discipline: is the selected candidate evaluated on a held-out test once, after search?
C4	Variance: repeated seeds or repeated evolution runs for headline numbers
C5	Overfitting check: validation-to-test gap or held-out transfer of the selected candidate
C6	Null control: random search, HPO or another non-LLM search at equal budget
C7	Budget: what is counted (iterations, GPU time, tokens, money)
C8	Baselines: are starting points and compared models tuned?
C9	Isolation: can the agent reach test data or evaluation code, and is metric gaming discussed?

3 Background

Search-space AutoML for recommenders. AutoML for recommenders searched for feature-interaction structures (AutoCTR [74], AutoFIS [44]), embedding dimensions (AutoEmb [105], AutoDim [104]), whole architectures with weight sharing (NASRec [100], DNSRec [99]) and loss functions (AutoLossGen [39]); Zheng et al. survey this line [106]. Each method can only return a design that its space contains. Evaluation is mostly validation logloss or AUC on public CTR data with standard splits, and the number of full evaluations is small, so the selection problems we discuss later are mild.

LLM program evolution and ML-engineering agents. FunSearch [72] and AlphaEvolve [60] pair an LLM that proposes code with an automatic evaluator and an archive of programs. Open systems followed: OpenEvolve [61], ShinkaEvolve [33] and DeepEvolve [46]. ML-engineering agents—AIDE [27], MLE-STAR [54], R&D-Agent [91], AIRA [77] and the AI Scientist [50, 88]—apply the same pattern to whole training pipelines, and recent work optimises the agent harness itself [35, 49, 87]. Their benchmarks contain at most one recommendation task: MLE-bench includes the H&M fashion-recommendation competition [9] and MLRC-Bench a next-product-recommendation task [101]. MLRC-Bench shows agents only the development split and scores the best development snapshot once on test, whereas MLGym's `validate` command returns test-set metrics during the session [56].

Generative recommenders. HSTU [94] and TIGER [68] recast recommendation as sequence generation over item IDs or semantic IDs, and OneRec unifies retrieval and ranking in one generative model that is further aligned by iterative preference optimisation on its own samples [16, 107, 108]. These models add new objects that a loop can evolve: tokenisers, codebooks, decoding procedures and generative losses.

Evaluation methodology. Tuned simple baselines beat many published neural recommenders [15, 70]; a loss change alone can reorder SASRec and BERT4Rec [31, 67]; sampled metrics are inconsistent with full-ranking metrics and can reverse model rankings [32]; leave-one-out splits that ignore the global timeline leak future information [26], and the split choice changes rankings [22, 51]. Libraries and open benchmarks such as RecBole [103], Elliot [1], DaisyRec [75], BARS [109] and FuxiCTR [110] standardise protocols for human-designed models. For adaptive reuse of a holdout, theory predicts overfitting [7, 17]; empirically, a meta-analysis of 120 Kaggle competitions found little [71]. We return to this tension in §6.

4 A Taxonomy

We place each system on two axes (Table 2).

Axis 1: the evolved object. **L0 search space**: a point in a human-written space (AutoML). **L1 configuration**: hyperparameters, fusion weights, retrieval budgets, thresholds. **L2 code**: model, loss, training, feature code. **L3 generative components**: semantic-ID construction, item vocabularies, decoding. **L4 policy of an agentic recommender**: its prompts, memory, skills or LLM weights.

Meta: the evolver itself—its prompts, skills, playbooks or a learned proposer.

Axis 2: the selection signal. **S1** offline validation with a separate test; **S2** an offline held-out set that is also the reported result; **S3** offline, split not stated; **S4** a proxy model or a small-scale screen; **S5** online A/B experiments; **S6** self-generated feedback (reflection, simulators, the model's own samples).

The two axes are not independent in practice. Code-level systems use S1, S3 and S5; component-level systems rely on S4 because full training is too expensive to run per candidate; policy-level systems use S6. Each signal has a characteristic way of failing: S1 overfits a reused validation set, S2 reports the selection bias as a gain, S3 cannot be assessed, S4 fails when the proxy does not transfer, S5 is vulnerable to short windows and to multiple testing over many rounds, and S6 can be circular when the judge and the judged come from the same model family. This is why we audit the selection signal rather than only the reported gain.

5 Systems

5.1 Configuration-level loops (L1)

L1 systems let an LLM change numbers or switches while the model code stays fixed. VirtualMLE tunes SASRec and HSTU through an execute–reflect loop with a hierarchical memory that transfers across datasets [8]. In production, CORAL allocates retrieval budgets and serving treatments [4], AgenticRecTune tunes fusion weights and thresholds across pre-ranking, ranking and re-ranking [84], Sortify adjusts the exchange rates of a sorting formula with Optuna inside an LLM meta-controller [14], GEARS selects segment-level treatment policies [92], and SR-Agent edits post-ranking diversity and exposure strategies through four bounded action types [89]. Most L1 systems in production select directly on online experiments.

5.2 Code-level loops in the open (L2)

Self-EvolveRec [30] runs an AlphaEvolve-style loop over the whole pipeline code of NCF, NGCF, SASRec and MoRec seeds; besides validation metrics it receives directional critiques from an LLM user simulator and from a diagnosis tool that evolves with the model. **RecHarness** [43] routes each round with Thompson sampling over human-defined "edit arms" (schedules, dropout, depth, pooling, features, loss) plus "jump" arms for structural changes, uses validation feedback only, evaluates the incumbent on test once, and caps compute at 43,200 GPU-seconds; HSTU is one of its five backbones. **EvoSkillRec** [38] evolves architectures as typed "skill genomes" of executable modules and promotes validated modules into a skill library. **RankEvolve** [12] composes Claude Code and Codex under an executable operating protocol in which the two agents review and repair each other's edits to HSTU, and introduces ExecML-HSTU, a benchmark of how often agent edits are executed correctly. **Population Evolution** [65] runs several evolution searches that share evidence across related datasets, screens candidates on small subsets, and promotes survivors to larger scales; it introduces RMD-Bench. AutoRecLab [5, 6] generates recommender experiments from natural-language descriptions; it automates experimentation rather than evolving a model, and we do not audit it.

5.3 Code-level loops in industry (L2)

Industrial systems share a two-stage design: an offline loop screens agent-written candidates, and online experiments decide launches. Google's loop for YouTube ranking [78] has Gemini personas propose optimiser, architecture and reward changes, promotes on validation loss and then on north-star A/B metrics, and reports LLM costs of about $20,000 over six months. RecEvolve [63] runs the full research loop on a production two-tower retrieval model. Meta's REA plans, launches and debugs ads-ranking experiments with up-front GPU budget approval [52]; A-MLE explores a portfolio of ads-ranking models with human checkpoints [20]; Auto-RecSys runs idea- and execution-evolution loops over multi-day training jobs [37]; and RankEvolve and Population Evolution above also come from Meta. Alibaba's EvoRec [53] pairs research and code agents with a skill evolver, and Astar [23] trains an 8B model on the commit history of an advertising system to propose the next evolution direction. AgentX, deployed at Kuaishou [34], and its successor AgentX-Model [90] run research lineages (reproduce, follow up, compose, diagnose) over production rankers, and a Kuaishou paper converts research-agent trajectories into auditable evidence [111]. Tencent's NOVA [48] adds verification gates that reject recommender-invalid edits such as target leakage, NetEase's AutoLR [98] reviews proposals with a multi-expert proposal council under a limited trial budget, and Tencent's RecSys Factory bounds agent autonomy to typed decision points in fixed pipelines [3]. A Trip.com co-scientist for hotel search ranking [83], LLM evolution of learning-to-rank pipelines (EvoRank [64]) and of lexical retrieval functions [59] are close relatives outside recommendation.

5.4 Components of generative recommenders (L3)

Generative recommenders expose objects that do not exist in ID-based models. Evolving Semantic Indexes runs LLM-driven program evolution over semantic-ID construction, collision handling, decoding and the generator's training objective, using a one-million-parameter proxy decoder as fitness [58]. AgenticTagger has an LLM refine a hierarchical item-descriptor vocabulary used as tokens [85], and a generative recommender can reassign its own item tokenisation during training [10]. Feedback-grounded policy discovery distils LLM-discovered natural-language policies into a semantic-ID generator [13]. No open system yet evolves TIGER- or OneRec-style components under a shared protocol.

5.5 Self-improving agentic recommenders (L4)

When the recommender is itself an LLM agent, the evolving object is its memory, skills, prompts or weights, and the feedback is mostly generated by the system. Memory and reflection methods update user and item agents (AgentCF [96], AgentCF++ [47], MACRec [81]), per-user policy skills (SAGER [76]), propagation modules optimised with textual gradients (RecNet [36]), and structured memories that are rewritten as interactions arrive (AtomRec [24], AMEM4Rec [57], STEAM [40], MARS [73], MemRec [11]).

Table 2: Taxonomy of self-evolving recommender systems: evolved object (rows) × selection signal (columns). Code: † open-source code verified.

	S1 offline val + test	S2/S3 offline, test reused or unstated	S4 proxy / screen	S5 online A/B (primary)
L0 search space	AutoFIS† [44], NASRec† [100], AutoCTR [74], AutoLossGen [39]			
L1 configuration	VirtualMLE† [8]	AutoLR [98]		CORAL [4], AgenticRecTune [84], Sortify [14], GEARS [92], SR-Agent [89]
L2 code	Self-EvolveRec† [30], RecHarness† [43], EvoSkillRec† [38]; *comparison:* EvoRank† (LTR) [64]	RankEvolve [12], EvoRec [53], A-MLE [20], Trajectories-to-Evidence [111], NOVA [48], AgentX-Model [90], Auto-RecSys [37], RecEvolve [63]; *comparison:* AI Co-Scientist (search) [83]	Population Evolution [65], Astar [23]	Google/YouTube [78], AgentX [34], Meta REA (blog) [52]
L3 components			Evolving Semantic Indexes [58], AgenticTagger [85]	
L4 policy (S6)	Memory/reflection: AgentCF† [96], AgentCF++† [47], SAGER [76], RecNet [36], AtomRec [24], MemRec† [11]; weights: OneRec [16], SPRec† [19], Rec-R1† [41], CoARS [80], RSIR† [97]; prompts and logic: Spotify [62]			
Meta	Learned direction proposer: Astar [23]; harness and skill evolution: AgentX SGPO [34], EvoRec skills [53], Auto-RecSys playbooks [37], AgenticRecTune skill hub [84]			

Weight-level self-improvement includes OneRec's iterative preference alignment on its own beam samples [16], self-play debiasing (SPRec [19]), RL against a recommender's metric (Rec-R1 [41]), co-evolving recommender and user agents (CoARS [80]), and recursive self-training on filtered synthetic sequences [21, 97], where progress can sit in earlier generations rather than the latest model [86]. Spotify improves a conversational recommendation agent's prompts and logic with a coding agent, evaluated on synthetic conversations [62]. Environments for training such agents include RecoWorld [45], Agent4Rec [95], RecAgent [79] and the verifiable τ-Rec benchmark [55].

5.6 Evolving the evolver (meta)

Several systems improve the loop itself: AgentX's SGPO applies semantic-gradient updates to its agents [34], EvoRec and AgenticRecTune distil skills from experiment memory [53, 84], Auto-RecSys evolves execution playbooks [37], and Astar learns the proposer from history [23]. These are the recommender counterparts of harness optimisation [35, 87].

6 How Are Self-Evolving Recommenders Evaluated? An Audit

Audited set. We audited the 22 recommendation systems at levels L1–L2 (and L1+L2 with meta-evolution) that select candidates with an evaluation loop, and five comparison points outside recommendation (learning to rank, search ranking, lexical retrieval, generative document retrieval and lifetime-value prediction). Fifteen of the 22 select candidates offline; six select on online experiments; Auto-RecSys reports only operational outcomes and makes no offline accuracy claim. Meta's REA is described only in a blog post and is not audited. Four borderline systems were read but not counted: RecSys Factory [3] (an operations platform with human approval of every change), AutoRecLab [5] (experiment generation), AURA [29] (stops at pull requests) and Spotify's prompt loop [62] (an L4 system). Table 3 gives the coding; counts below are generated from the released data file.

F1: Almost nothing is evaluated with full-catalogue ranking. Of the 22 systems, two state a full-catalogue ranking evaluation (RankEvolve on HSTU and Population Evolution). Two rank against sampled negatives: Self-EvolveRec and RecHarness pair each test item with 99 random negatives. Four report ranking metrics without saying what the candidate set is; for RecEvolve, the fact that shrinking the batch inflated its metrics (F7) suggests in-batch negatives. Six use AUC or a loss as the offline metric, six select on online experiments, and two report no usable offline metric.

F2: The selection split is usually not stated, and sometimes it is the test set. Of the 15 systems with an offline selection loop, five name a validation split (Self-EvolveRec, RecHarness, VirtualMLE, EvoSkillRec, and Google's validation loss), one selects on training loss and a learned reward model (Astar), and eight do not say which data accept or reject candidates. RankEvolve selects on test data: its persistent leaderboard is the canonical full-test NDCG@10 [12]. Population Evolution selects candidates on source instances and reports on others, but calibrates the shared baseline configuration of its transfer targets on their released test split [65]. Both disclose these choices, which is to their credit. Outside recommendation, the hotel-search co-scientist keeps or reverts each change on the same held-out metric that it reports [83].

F3: Test-once is rarely stated. Three of the 15 offline loops state that the selected candidate was evaluated on held-out test data once

Table 3: Evaluation audit (full text, 21 October 2026). C1 metric: Full = full-catalogue ranking; Sampled = ranking against sampled negatives; Rank? = ranking metric with the candidate set not stated; AUC = AUC/logloss/loss proxy; A/B = online only; ? = no metric stated. C2 selection signal. C8 baselines: prod. = production model; pub. = published anchors. C4: P = repeats without spread, repeats in an ablation only, or a re-run rule. C9: P = part of the loop constrained (fixed evaluation code, gates, whitelists, sandboxing); G = metric inflation or target leakage by the loop reported; Y+G = isolation stated and leakage bugs caught by an audit. Y = yes, N = no, P = partial, NS = not stated, – = not applicable.

System	Level	C1 metric	C2 select	C3 test once	C4 repeats	C5 gap	C6 null	C8 baselines	C9 isolation	A/B	Code
Self-EvolveRec [30]	L2	S99	val	NS	N	P	P	N	NS	N	Y
RecHarness [43]	L2	S99	val	Y	P	N	P	N	P	Y	Y
Google/YouTube [78]	L1+L2	AUC	val	NS	P	P	N	prod.	NS	Y	N
EvoRec [53]	L2	LOO?	NS	NS	N	N	P	NS	NS	Y	N
VirtualMLE [8]	L1	LOO?	val	Y	P	P	Y	Y	Y	N	Y
NOVA [48]	L2	AUC	NS	NS	N	N	Y	prod.	NS	Y	N
AgentX [34]	L2+meta	A/B	A/B	–	N	P	N	prod.	NS	Y	N
AgentX-Model [90]	L2	AUC	NS	NS	P	N	P	prod.	P	Y	N
Auto-RecSys [37]	L2+meta	?	NS	NS	N	N	N	prod.	NS	N	N
AutoLR [98]	L1+L2	?	NS	NS	N	N	N	prod.	P	Y	N
CORAL [4]	L1	A/B	A/B	–	N	N	N	prod.	–	Y	N
AgenticRecTune [84]	L1+meta	A/B	A/B	–	N	N	N	prod.	–	Y	N
Sortify [14]	L1	A/B	A/B	–	N	P	N	prod.	–	Y	N
GEARS [92]	L1	A/B	A/B	–	N	P	N	prod.	–	Y	N
RecEvolve [63]	L2	S99	NS	NS	N	N	N	prod.	G	Y	N
Astar [23]	L2+meta	LOO?	loss	NS	N	P	P	prod.	NS	Y	N
A-MLE [20]	L1+L2	AUC	NS	NS	P	N	P	prod.	G	N	N
SR-Agent [89]	L1	A/B	A/B	–	N	P	N	prod.	P	Y	N
EvoSkillRec [38]	L2	AUC	val	Y	N	Y	Y	N	P	N	Y
RankEvolve-HSTU [12]	L1+L2	Full	test	N	N	P	P	pub.	G	N	later
Population Evolution [65], Astar [23]	L2	Full	test	N	N	Y	P	Y	NS	N	later
Trajectories-to-Evidence [111]	L2	AUC	NS	NS	N	P	P	prod.	P	Y	N
Comparison points outside recommendation											
EvoRank (LTR) [64]	L2	Full	val	Y	Y	Y	Y	Y	Y	N	Y
AI Co-Scientist (search) [83]	L2	AUC	test	N	N	N	N	P	G	N	N
RankEvolve-IR (lexical) [59]	L2	Full	NS	Y	N	Y	P	N	N	N	Y
Evolving Semantic Indexes (GR) [58]	L3	Full	val	Y	P	Y	N	seed	Y	N	N
AgentLTV (LTV) [82]	L2	other	NS	NS	N	N	P	NS	N	N	Y

after the search (RecHarness, VirtualMLE, EvoSkillRec); RankEvolve reports on the test leaderboard it selects on; Population Evolution partly; ten do not say.

F4: No headline result comes with its spread over repeated sessions. RecHarness and VirtualMLE average three runs without reporting a standard deviation; Google runs six independent sessions with confidence intervals, but only for an ablation; A-MLE has a rule to re-run when within-run variance is high. Several papers state that their gains are within the resolution of a single run: EvoSkillRec notes margins "on the order of 10^{-4} AUC, which a single evolution run cannot reliably separate from run-to-run variation" [38]; AgentX-Model reports run-to-run standard deviations of 0.0010–0.0022 AUC while counting an experiment as above baseline if it exceeds it by more than 10^{-6} [90]; and AutoLR, in its audited configuration, promoted candidates after a single run with noise calibration disabled; the authors themselves identify the resulting risk—each promoted model becomes the baseline for all later comparisons—and name it the "KEEP ratchet" [98].

F5: Where transfer is measured, selected gains shrink, sometimes to zero. Two core systems report an explicit held-out check of their candidates. In EvoSkillRec, five candidates lie on the validation Pareto front, "but only P1 (round 19) retains both improvements on the test set" [38]; the candidate it selects is P1. In Population Evolution, a ranking candidate gains 10.66% on a peer dataset and loses 0.79% at larger scale, and two of three peer-screened designs keep their gains; sharing evidence across searches raises the larger-scale pass rate for ranking from 11/60 to 18/60 [65]. Kuaishou's trajectory analysis finds that for 22 of 30 methods the last round falls below an earlier best [111], so the final state of a loop is not necessarily its best. The comparison points tell the same story: in EvoRank, the gains of the first (loss-evolution) campaign "were almost entirely fitness noise" on 60k held-out queries, and about 70% of the second campaign's gains survived [64]; in Evolving Semantic Indexes, a proxy decoder ranks semantic-ID programs well (Spearman $\rho = 0.90$), but once the generator's training objective

may change, "the proxy becomes not only less informative but misleading", with five programs ranked inversely on proxy and target ($\rho = -0.80$) [58].

F6: Null controls are rare. Three core systems compare against a non-LLM search at an equal or favourable budget: VirtualMLE (grid search and Bayesian optimisation), NOVA (Optuna-TPE) and EvoSkillRec (NASRec, alongside OpenEvolve); in all three the LLM loop is reported to win. Nine have partial controls (unmatched budgets, a random arm router that still uses LLM edits, cross-LLM swaps, or replay-only comparisons), and ten have none. No core system runs an equal-budget random search over its own edit space. Where random selection was run, it was competitive: in AgentX-Model's historical replay, which randomises the choice of experiments rather than the edits, "Uniform Random's mean exceeds Fixed, Greedy, and Joint" policies [90], and in EvoRank's loss campaign an equal-budget random search transferred as well or better ($p = 0.03$) [64]. AutoLR states plainly that it "does not establish superiority over random, greedy, or direct LLM-based routing under a matched trial budget" [98].

F7: Agents find shortcuts in the evaluation; humans and monitoring, not the acceptance rule, catch them. Two core systems and two comparison points report metric inflation or target leakage produced by the loop itself. RecEvolve's agent "discovered that drastically reducing the batch size (from 8k down to 1k) artificially inflated proxy metrics"; a human caught it and rolled it back [63]. On HSTU, RankEvolve's agents produced a genre feature that leaked the target (training loss collapsed to zero while evaluation spiked), a multi-position evaluation that inflated NDCG@10 from 0.2192 to 0.2577, and DPO negatives derived from the target. These surfaced as anomalous trajectories on the leaderboard and were diagnosed before being reported as gains, but the best agent pair still had a 10.4% silent critical-defect rate on ExecML-HSTU, and a single Claude Code pass 35.4% [12]. Agent reliability is a related problem: A-MLE's agent promoted candidates on a single seed until a re-run rule reduced this, and weaker models "pretend to have completed tasks" [20]. EvoRank's audit caught three target-encoding leakage bugs [64], and the search co-scientist produced "metric-winning bugs (e.g., silently increasing model size)" [83]. Among the borderline systems, RecSys Factory found a join bug that "silently corrupted all downstream training samples" and AURA's agent fabricated 904 identifiers [3, 29]. Only VirtualMLE states that the test set is hidden from the agent and the data pipeline immutable; ten systems constrain part of the loop (verification gates, whitelists, fixed evaluation code, frozen evaluation logic, isolated workspaces, sandboxed modules, bounded actions), and five say nothing about it.

F8: Baselines, budgets and openness. The largest relative gains are measured against weak starting points: RecHarness improves average HR@10 by 85.85% over a cold-start GRU4Rec template and by 12.58% over HSTU, and also lists published numbers well above its cold-start base [43]; Self-EvolveRec configures all models with the same untuned hyperparameters [30]. Only VirtualMLE and Population Evolution describe tuning their baselines. Budgets are reported in incompatible units (iterations, GPU-seconds, GPU-hours, ideas, tokens, dollars, yuan); twelve of 22 report LLM tokens or cost. Four of 22 release code, two promise to, and five do not name the LLM they use.

Table 4: Reporting standard for self-evolving recommenders.

R1	Report full-catalogue ranking metrics (sampled metrics only as secondary), with the candidate set stated.
R2	Name the split or signal that accepts candidates; keep the test split out of every decision.
R3	Evaluate the finally selected candidate on test once, and report its validation score next to it.
R4	Repeat the whole evolution session ($\geq$ 5 seeds, or as many as affordable) and report every session's final test score with the median and range.
R5	Run a null control at equal trial and GPU budget: random search or HPO over the same configuration or edit-arm space, or, for open-ended edits, the same proposer with candidates accepted at random.
R6	Report the budget as a card: number of trials, GPU time, LLM tokens and cost, wall-clock, and the cost of training a learned proposer.
R7	Start from and compare against tuned baselines; report how they were tuned.
R8	State what the agent can read and write: data, labels, evaluation code; report gaming incidents found.
R9	For online gates: duration, traffic share, pre-specified metric and guardrails, confidence intervals, the number of rounds tested with a correction rule across them, and a fresh pre-registered confirmation of the final candidate.

Online gates. Fifteen of the 22 systems report online experiments. These are strong evidence when well designed, and they are the main safeguard of the industrial loops. Seven state the experiment duration and eight report significance or confidence intervals; none corrects for multiple testing across the many rounds a loop runs (AutoLR discusses the problem but applies no correction). AutoLR's launch-review lifts are, in its own words, "descriptive and are not pooled treatment effects" [98]; Sortify chose its frozen round on the same online signal, then re-tested it in a fresh seven-day experiment [14], a practice we recommend.

Does selection really overfit? A meta-analysis of 120 Kaggle competitions found little overfitting to public leaderboards [71], which cautions against assuming that reuse of a validation set must inflate results. The setting differs in ways that matter: Kaggle limits submissions and teams submit few, diverse models, whereas an agent loop evaluates hundreds of correlated candidates on one split without a cap, keeps the maximum, and controls the code that produces the score. The direct measurements we found (F5) show that some selected gains shrink or vanish on transfer, while others survive. The safe conclusion is not that self-evolving recommenders do not work, but that the published evidence cannot yet tell how much of their gain is real.

7 A Reporting Standard and a Shared Protocol

The audit suggests that most of the problems are cheap to report; they are unreported or unenforced. We propose a reporting standard (Table 4) and a protocol that enforces the parts of it that can be enforced in software.

Cost of compliance. The expensive item is R4, which multiplies the cost of a session. Published per-session costs suggest it is affordable for academic settings: VirtualMLE reports $1.5–4.8 of LLM cost per session [8], EvoRank about ten dollars for fifty iterations [64], and NOVA about $29 per task [48]; training cost dominates and is shared with R5. For industrial loops whose candidates take hundreds of GPU-hours [37], R4 can be applied to a replay or a reduced-scale proxy, as AgentX-Model does for scheduling policies [90].

recforge. We built recforge [69], an open benchmark that implements R1–R3, R5, part of R6, and part of R8 for sequential recommendation. Its design choices follow from the audit:

- *One protocol.* Leave-one-out per user (last item test, second-to-last validation), full-catalogue ranking with the user's history removed, and HR and NDCG at 10 and 20 and MRR@100. Results from different protocol versions are never put in one table, and every result is keyed by the source hash of the method, its configuration and the code commit.
- *A sealed test at the API level.* Method code runs in its own process and receives only a training view of the data; asking it for a validation or test target raises an error, and the method returns ranked lists that the parent process scores. Sealed scores and session state are ordinary files, so an agent with file-system access could read test scores or alter the budget; until the planned container mode, the seal holds only against code that uses the API.
- *Budgeted evolve sessions.* A session has a trial and time budget. Each trial returns validation metrics only; test metrics are computed and sealed. A single `finalize` reveals the test metrics of exactly one trial, publishes it to the leaderboard with the session's budget attached, and closes the session.
- *Evolvers on equal terms.* Any evolver drives a session through the same interface: an equal-budget random search (the R5 null control), a wrapper that runs an external coding agent such as Claude Code or Codex, and an OpenEvolve adapter. Each run reports the validation-to-test gap of its finalized trial. These evolvers are implemented and tested on synthetic data but not yet exercised in this survey.
- *Baselines and data.* Popularity, ItemKNN, BPR, SASRec [28], HSTU [94] and TIGER [68] are built in, with MovieLens-1M, Amazon 2014 Beauty/Sports/Toys, MIND news, and the Kuaishou OpenOneRec release [107].

recforge does not yet satisfy the rest of the standard, and we list the gaps rather than claim them: the training view protects the API but not the file system, so code that opens the raw data files or the sealed score files can still read targets or test results (a container mode is planned); LLM tokens and cost are not recorded; repeated sessions are not aggregated automatically; there is no timeline-aware split [26]; and the OpenEvolve adapter has not yet been run end to end. We report no evolver results in this survey.

8 Open Problems

Isolation without crippling the agent. Code-level agents need to read data and change training code, and that is exactly the access that lets them leak targets or change what the metric measures (F7). Verification gates (NOVA), fixed evaluation code (AgentX-Model), immutable pipelines (VirtualMLE) and cross-agent review (RankEvolve) each close part of the gap. A principled design would give the agent a process that physically cannot reach evaluation labels or code, and would audit every metric-relevant change, such as batch size for in-batch metrics.

Proxies for expensive candidates. Industrial candidates take hours to hundreds of GPU-hours, so loops screen on proxies: small decoders [58], data subsets and peer datasets [65], training-loss curves and learned reward models [23]. The evidence shows that proxy fidelity depends on what is evolved. Characterising when a proxy can be trusted, and spending budget to re-check proxies during search, is open.

Budget-normalised comparison of evolvers. There is no common unit of cost and no common task, so the field cannot say whether bandit routing, island evolution, research-agent pairs or learned proposers are better. A shared benchmark with a budget card (R6) and an equal-budget random baseline (R5) would allow the first such comparison.

Evolving generative recommenders. Semantic-ID construction, codebook collisions, constrained decoding and preference-alignment losses are new design dimensions with little human prior and large effects. They are natural targets for code-level evolution, and almost untouched (§5).

From offline wins to online value. RecEvolve observes that offline evaluations often show "win-win" results while online tests show trade-offs [63]. Loops that select on offline metrics inherit the gap; loops that select on online experiments face multiple testing and slow iteration. Combining the two with pre-registered online confirmation of the final candidate, as Sortify does, is a starting point.

Circularity at the policy level. L4 systems are often judged by simulators from the same model family as the agent being improved. Held-out simulators [30], verifiable rewards [55] and reward-model benchmarks [93] are partial answers.

9 Threats to Validity

Coverage. The field is months old and grows weekly; our cut-off is 21 October 2026. Many systems are arXiv preprints whose versions change; we coded the version available on that date. Generic keyword search performed poorly, so coverage relies on citation-graph search and may miss systems that neither cite nor are cited by the core set.

Coding. The audit was coded from full text with verbatim evidence and then checked against the original papers by an adversarial second reading, which corrected 12 cells in nine rows; we have not yet measured agreement between two independent coders. "Not stated" does not mean "not done": authors may have followed a practice without reporting it. We release the evidence so that authors and readers can correct individual cells, and we will update the table.

Industrial systems. Offline hygiene criteria are less important for systems whose final gate is a well-designed online experiment. We

therefore report online-gate design separately (R9) and do not treat a missing offline test split as a defect when no offline claim is made.

Conflict of interest. The authors built recforge, the benchmark proposed in §7. recforge was developed alongside this survey, so its design reflects our reading of the literature; we list its gaps next to its features, and the audit itself does not depend on it.

10 Conclusion

Self-evolving recommenders moved in one year from an idea to production systems at many large platforms. The loops are inventive, and several report online gains under well-designed experiments. The offline evidence is weaker than the headlines suggest: selection splits are mostly unstated, test-once and repeated sessions are rarely reported, equal-budget random search is absent, and the few papers that check transfer find that some selected gains do not survive. Agents also find shortcuts in the evaluation, which humans and monitoring rather than the acceptance rules catch. Most of this is cheap to report. We hope the reporting standard in Table 4, the released audit data, and an open protocol such as recforge make it the norm, so that the next comparison of evolvers can be made on equal terms.

References

[1] Vito Walter Anelli, Alejandro Bellogín, Antonio Ferrara, Daniele Malitesta, Felice Antonio Merra, Claudio Pomo, Francesco Maria Donini, and Tommaso Di Noia. 2021. Elliot: a Comprehensive and Rigorous Framework for Reproducible Recommender Systems Evaluation. arXiv:2103.02590. arXiv:2103.02590 https://arxiv.org/abs/2103.02590

[2] Huan ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Qihan Ren, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. 2025. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. arXiv:2507.21046. arXiv:2507.21046 https://arxiv.org/abs/2507.21046

[3] Dongyang Ao, Kaixiang Fang, and Shijie Xu. 2026. RecSys Factory: Bounding LLM Agent Autonomy to Decision Points in the Industrial Recommender Lifecycle. arXiv:2608.11241. arXiv:2608.11241 https://arxiv.org/abs/2608.11241

[4] Muhammad Rafay Azhar, Yuhang Zhou, Gilbert Jiang, Yuchen Wang, Rahul Sharma, Matthew DeSousa, Jiayi Liu, Xin Guo, Lizhu Zhang, and Xiangjun Fan. 2026. CORAL: An LLM-Native Harness for Production Recommender Systems. arXiv:2609.02730. arXiv:2609.02730 https://arxiv.org/abs/2609.02730

[5] Moritz Baumgart, Philipp Meister, Justus Krell, Michael Schmidt, Bela Gipp, and Joeran Beel. 2026. AutoRecLab: Describe the Experiment, Get the Code! arXiv:2609.21863. arXiv:2609.21863 https://arxiv.org/abs/2609.21863

[6] Joeran Beel, Bela Gipp, Tobias Vente, Moritz Baumgart, and Philipp Meister. 2025. From AutoRecSys to AutoRecLab: A Call to Build, Evaluate, and Govern Autonomous Recommender-Systems Research Labs. arXiv:2510.18104. arXiv:2510.18104 https://arxiv.org/abs/2510.18104

[7] Avrim Blum and Moritz Hardt. 2015. The Ladder: A Reliable Leaderboard for Machine Learning Competitions. arXiv:1502.04585. arXiv:1502.04585 https://arxiv.org/abs/1502.04585

[8] Shiteng Cao, Jingwen Liu, Junda She, and Zhiheng Li. 2026. VirtualMLE: A Virtual ML Engineer that Optimizes Sequential Recommenders. arXiv:2606.03221. arXiv:2606.03221 https://arxiv.org/abs/2606.03221

[9] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. 2024. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. arXiv:2410.07095. arXiv:2410.07095 https://arxiv.org/abs/2410.07095

[10] Runjin Chen, Mingxuan Ju, Ngoc Bui, Dimosthenis Antypas, Stanley Cai, Xiaopeng Wu, Leonardo Neves, Zhangyang Wang, Neil Shah, and Tong Zhao. 2024. Enhancing Item Tokenization for Generative Recommendation through Self-Improvement. arXiv:2412.17171. arXiv:2412.17171 https://arxiv.org/abs/2412.17171

[11] Weixin Chen, Yuhan Zhao, Jingyuan Huang, Zihe Ye, Clark Mingxuan Ju, Tong Zhao, Neil Shah, Li Chen, and Yongfeng Zhang. 2026. MemRec: Collaborative Memory-Augmented Agentic Recommender System. arXiv:2601.08816. arXiv:2601.08816 https://arxiv.org/abs/2601.08816

[12] Zheng Chen, Linfeng Liu, Hong Li, and Hong Yan. 2026. RankEvolve: A Reliable Multi-Agent Auto-Research Harness for Evolving Ranking Models. arXiv:2609.39551. arXiv:2609.39551 https://arxiv.org/abs/2609.39551

[13] Zhi Chen, Minmao Wang, Xingchen Liu, Haoqiang Liang, Huihuang Lin, Likang Wu, Hongke Zhao, Yulong Wang, Shijie Yi, Fei Pan, and Peng Jiang. 2026. From Understanding to Action: Feedback-Grounded Policy Discovery for Generative Recommendation. arXiv:2607.27789. arXiv:2607.27789 https://arxiv.org/abs/2607.27789

[14] Yin Cheng, Liao Zhou, Xiyu Liang, Dihao Luo, Tewei Lee, Kailun Zheng, Weiwei Zhang, Mingchen Cai, Jian Dong, and Andy Zhang. 2026. Let the Agent Steer: Closed-Loop Ranking Optimization via Influence Exchange. arXiv:2603.27765. arXiv:2603.27765 https://arxiv.org/abs/2603.27765

[15] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. 2019. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. arXiv:1907.06902. arXiv:1907.06902 https://arxiv.org/abs/1907.06902

[16] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. arXiv:2502.18965. arXiv:2502.18965 https://arxiv.org/abs/2502.18965

[17] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. 2015. Generalization in Adaptive Data Analysis and Holdout Reuse. arXiv:1506.02629. arXiv:1506.02629 https://arxiv.org/abs/1506.02629

[18] Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. 2025. A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems. arXiv:2508.07407. arXiv:2508.07407 https://arxiv.org/abs/2508.07407

[19] Chongming Gao, Ruijun Chen, Shuai Yuan, Kexin Huang, Yuanqing Yu, and Xiangnan He. 2024. SPRec: Self-Play to Debias LLM-based Recommendation. arXiv:2412.09243. arXiv:2412.09243 https://arxiv.org/abs/2412.09243

[20] Erwin Gao, Vinodh Kumar Sunkara, Jingyi Guan, Qinjin Jia, Hangjun Xu, Xiang Ji, Sherman Wong, Surya Teja Chavali, Pratik Vaishnavi, Aryan Pandhi, Xiaoyu Deng, Zhaodong Wang, Samarth Inani, Fan Yang, Jakob Moberg, Zoe Zu, Nicolas Bievre, Sami Khenissi, Amit Jaspal, Ehsan Fakharizadi, Srinidhi Viswanathan, Dorothy Sun, Abishek Vanam, Sneha Iyer, Sheela Yadawad, Wenjie Chen, Gaby Nahum, Junhua Gu, Peter Chu, Yucheng Liu, Xin Zhao, Vitor Cid, Chaorong Chen, Vijay Pappu, Ashwin Kumar, Wenlin Chen, Ben Schulte, Deepak Chandra, and Ritwik Tewari. 2026. Agentic ML Exploration (A-MLE) for Ads Ranking. arXiv:2609.08248. arXiv:2609.08248 https://arxiv.org/abs/2609.08248

[21] Tonmoy Hasan, Taylor Foust, Shao Tang, Leonardo Neves, Aman Gupta, Hiroto Udagawa, Helder Dias, Daniel Silva, and Rohan Ramanath. 2026. From Valid to Useful: Post-Verification Acquisition for Recursive Self-Improving Recommendation. arXiv:2610.04302. arXiv:2610.04302 https://arxiv.org/abs/2610.04302

[22] Balázs Hidasi and Ádám Tibor Czapp. 2023. Widespread Flaws in Offline Evaluation of Recommender Systems. arXiv:2307.14951. arXiv:2307.14951 https://arxiv.org/abs/2307.14951

[23] Jinxin Hu, Hao Deng, Haibo Xing, Lingyu Mu, Muyu Zou, Weiqin Yang, Sirui Chen, Bohao Wang, Zhezheng Hao, Hao Zhang, Zulong Chen, Shizhun Wang, Yu Zhang, Xiaoyi Zeng, and Jiawei Chen. 2026. Astar: Learning to Propose Evolution Directions for Self-Evolving Industrial AI Systems. arXiv:2608.27287. arXiv:2608.27287 https://arxiv.org/abs/2608.27287

[24] Peiyu Hu, Weihai Lu, Siying Gu, Zhuodong Liu, Zhaokai Luo, Yuean Niu, Zhiyong Wang, and Jia Wang. 2026. AtomRec: Evolving Atomic Memory for Agentic Recommendation. arXiv:2609.04882. arXiv:2609.04882 https://arxiv.org/abs/2609.04882

[25] Chengkai Huang, Hongtao Huang, Tong Yu, Kaige Xie, Junda Wu, Shuai Zhang, Julian Mcauley, Dietmar Jannach, and Lina Yao. 2025. A Survey of Foundation Model-Powered Recommender Systems: From Feature-Based, Generative to Agentic Paradigms. arXiv:2504.16420. arXiv:2504.16420 https://arxiv.org/abs/2504.16420

[26] Yitong Ji, Aixin Sun, Jie Zhang, and Chenliang Li. 2020. A Critical Study on Data Leakage in Recommender System Offline Evaluation. arXiv:2010.11060. arXiv:2010.11060 https://arxiv.org/abs/2010.11060

[27] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. 2025. AIDE: AI-Driven Exploration in the Space of Code. arXiv:2502.13138. arXiv:2502.13138 https://arxiv.org/abs/2502.13138

[28] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. arXiv:1808.09781. arXiv:1808.09781 https://arxiv.org/abs/1808.09781

[29] SungGeun Kim, Abhinav Narain, and Daniel Nemirovsky. 2026. AURA: Agentic Diagnosis and Refinement for Production Recommender Systems at Scale.

arXiv:2609.16625. arXiv:2609.16625 https://arxiv.org/abs/2609.16625

[30] Sein Kim, Sangwu Park, Hongseok Kang, Wonjoong Kim, Jimin Seo, Yeonjun In, Kanghoon Yoon, Hyunsik Jeon, and Chanyoung Park. 2026. Self-EvolveRec: Self-Evolving Recommender Systems with LLM-based Directional Feedback. arXiv:2602.12612. arXiv:2602.12612 https://arxiv.org/abs/2602.12612

[31] Anton Klenitskiy and Alexey Vasilev. 2023. Turning Dross Into Gold Loss: is BERT4Rec really better than SASRec? arXiv:2309.07602. arXiv:2309.07602 https://arxiv.org/abs/2309.07602

[32] Walid Krichene and Steffen Rendle. 2020. On Sampled Metrics for Item Recommendation. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. https://research.google/pubs/on-sampled-metrics-for-item-recommendation/

[33] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. 2025. ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution. arXiv:2509.19349. arXiv:2509.19349 https://arxiv.org/abs/2509.19349

[34] Changxin Lao, Fei Pan, Guozhuang Ma, Han Li, Huihuang Lin, Jijun Shi, Kangzhi Zhao, Kun Gai, Mo Zhou, Qinqin Zhou, Quan Chen, Ruochen Yang, Shifu Bie, Shijie Yi, Shuang Yang, Shuo Yang, Wenhao Li, Wentao Xie, Xiao Lv, Xuming Wang, Yijun Wang, Yiming Chen, Yusheng Huang, Zhongyuan Wang, Zibo Zhao, Zijie Zhuang, Baoning Xia, Chao Liu, Chaoyi Ma, Chubo He, Dawei Cong, Feng Jiang, Gang Wang, Guilin Xia, Hanwen Xu, Jiahong Xie, Jiahui Qiao, Jian Liang, Jiangfan Yue, Jing Wang, Jinghan Yang, Jinghui Jia, Kan Qin, Lei Wang, Ming Li, Peilin Song, Pengbo Xu, Qiang Luo, Ruiming Tang, Shiyang Liu, Shuxian Jin, Tao Wang, Tao Zhang, Xiang Gao, Xianghan Li, Yingsong Luo, Yiwen Ning, Yongcheng Liu, Yueyang Liu, Yuan Guo, Zhaojie Liu, and Zhenkai Cui. 2026. AgentX: Towards Agent-Driven Self-Iteration of Industrial Recommender Systems. arXiv:2606.26859. arXiv:2606.26859 https://arxiv.org/abs/2606.26859

[35] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-Harness: End-to-End Optimization of Model Harnesses. arXiv:2603.28052. arXiv:2603.28052 https://arxiv.org/abs/2603.28052

[36] Bingqian Li, Xiaolei Wang, Junyi Li, Weitao Li, Long Zhang, Sheng Chen, Wayne Xin Zhao, and Ji-Rong Wen. 2026. RecNet: Self-Evolving Preference Propagation for Agentic Recommender Systems. arXiv:2601.21609. arXiv:2601.21609 https://arxiv.org/abs/2601.21609

[37] Ming Li, Dai Li, Xuying Ning, Bo Sun, Rui Li, Yi Zhang, Silvia Gong, Xuan Cao, Rui Li, Cornelia Carapcea, Qunshu Zhang, Zhigang Wang, Yinglong Xia, and Andy Wang. 2026. Auto-RecSys: Harnessing Autonomous Research Agents for Industry-Scale Recommender System. arXiv:2609.10922. arXiv:2609.10922 https://arxiv.org/abs/2609.10922

[38] Xiaopeng Li, Kuo Cai, Bo Chen, Wenlin Zhang, Mengyang Ma, Yingyi Zhang, Zichuan Fu, Yu Yang, Qidong Liu, Yiyu Wang, Ruiming Tang, Wenwu Ou, Jiang Wu, Zhanbo Xu, and Xiangyu Zhao. 2026. EvoSkillRec: Skill-Genome Evolution for Recommender Architecture Discovery. arXiv:2609.34552. arXiv:2609.34552 https://arxiv.org/abs/2609.34552

[39] Zelong Li, Jianchao Ji, Yingqiang Ge, and Yongfeng Zhang. 2022. AutoLossGen: Automatic Loss Function Generation for Recommender Systems. arXiv:2204.13160. arXiv:2204.13160 https://arxiv.org/abs/2204.13160

[40] Yuxin Liao, Le Wu, Min Hou, Yu Wang, Han Wu, and Meng Wang. 2026. From Atom to Community: Structured and Evolving Agent Memory for User Behavior Modeling. arXiv:2601.16872. arXiv:2601.16872 https://arxiv.org/abs/2601.16872

[41] Jiacheng Lin, Tian Wang, and Kun Qian. 2025. Rec-R1: Bridging Generative Large Language Models and User-Centric Recommendation Systems via Reinforcement Learning. arXiv:2503.24289. arXiv:2503.24289 https://arxiv.org/abs/2503.24289

[42] Xinyu Lin, Yashar Deldjoo, Sunhao Dai, Honghui Bao, Xiaopeng Ye, Fatemeh Nazary, Wenjie Wang, Tommaso Di Noia, Jun Xu, and Tat-Seng Chua. 2026. Autonomous Information Seeking: A Roadmap for Agentic Recommender Systems. arXiv:2607.04433. arXiv:2607.04433 https://arxiv.org/abs/2607.04433

[43] Haoran Ling, Yuecheng Li, Zeyu Song, Jing Yao, Shuwen Kang, Chi Lu, Wenjin Wu, and Peng Jiang. 2026. RecHarness: A Bandit-Routed Agentic Harness for Self-Evolving Recommender Systems. arXiv:2607.29241. arXiv:2607.29241 https://arxiv.org/abs/2607.29241

[44] Bin Liu, Chenxu Zhu, Guilin Li, Weinan Zhang, Jincai Lai, Ruiming Tang, Xiuqiang He, Zhenguo Li, and Yong Yu. 2020. AutoFIS: Automatic Feature Interaction Selection in Factorization Models for Click-Through Rate Prediction. arXiv:2003.11235. arXiv:2003.11235 https://arxiv.org/abs/2003.11235

[45] Fei Liu, Xinyu Lin, Hanchao Yu, Mingyuan Wu, Jianyu Wang, Qiang Zhang, Zhuokai Zhao, Yinglong Xia, Yao Zhang, Weiwei Li, Mingze Gao, Qifan Wang, Lizhu Zhang, Benyu Zhang, and Xiangjun Fan. 2025. RecoWorld: Building Simulated Environments for Agentic Recommender Systems. arXiv:2509.10397. arXiv:2509.10397 https://arxiv.org/abs/2509.10397

[46] Gang Liu, Yihan Zhu, Jie Chen, and Meng Jiang. 2025. Scientific Algorithm Discovery by Augmenting AlphaEvolve with Deep Research. arXiv:2510.06056. arXiv:2510.06056 https://arxiv.org/abs/2510.06056

[47] Jiahao Liu, Shengkang Gu, Dongsheng Li, Guangping Zhang, Mingzhe Han, Hansu Gu, Peng Zhang, Tun Lu, Li Shang, and Ning Gu. 2025. AgentCF++: Memory-enhanced LLM-based Agents for Popularity-aware Cross-domain Recommendations. arXiv:2502.13843. arXiv:2502.13843 https://arxiv.org/abs/2502.13843

[48] Shaohua Liu, Liang Fang, Yilong Sun, Shudong Huang, Qingsong Luo, Shaoxin Liu, Xiaoyang Chen, Dongqiang Liu, Chuangang Ma, Zhenzhen Chai, Henghuan Wang, Shijie Quan, Changyuan Cui, Zhangbin Zhu, Peng Chen, Wei Xu, Lei Xiao, Haijie Gu, and Jie Jiang. 2026. NOVA: A Verification-Aware Agent Harness for Architecture Evolution in Industrial Recommender Systems. arXiv:2606.27243. arXiv:2606.27243 https://arxiv.org/abs/2606.27243

[49] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. 2026. AutoHarness: improving LLM agents by automatically synthesizing a code harness. arXiv:2603.03329. arXiv:2603.03329 https://arxiv.org/abs/2603.03329

[50] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv:2408.06292. arXiv:2408.06292 https://arxiv.org/abs/2408.06292

[51] Zaiqiao Meng, Richard McCreadie, Craig Macdonald, and Iadh Ounis. 2020. Exploring Data Splitting Strategies for the Evaluation of Recommendation Models. arXiv:2007.13237. arXiv:2007.13237 https://arxiv.org/abs/2007.13237

[52] Meta Engineering. 2026. Ranking Engineer Agent (REA): An Autonomous AI System Accelerating Meta Ads Ranking Innovation. Engineering at Meta blog, 17 March 2026. https://engineering.fb.com/2026/03/17/developer-tools/ranking-engineer-agent-rea-autonomous-ai-system-accelerating-meta-ads-ranking-innovation/

[53] Lingyu Mu, Hao Deng, Haibo Xing, Jinxin Hu, Yu Zhang, and Xiaoyi Zeng. 2026. EvoRec: Self Evolving Agentic Recommender Systems. arXiv:2606.28368. arXiv:2606.28368 https://arxiv.org/abs/2606.28368

[54] Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Sercan Ö. Arık, and Tomas Pfister. 2025. MLE-STAR: Machine Learning Engineering Agent via Search and Targeted Refinement. arXiv:2506.15692. arXiv:2506.15692 https://arxiv.org/abs/2506.15692

[55] Bharath Sivaram Narasimhan and Karthik R Narasimhan. 2026. τ-Rec: A Verifiable Benchmark for Agentic Recommender Systems. arXiv:2606.10156. arXiv:2606.10156 https://arxiv.org/abs/2606.10156

[56] Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, Dieuwke Hupkes, Ricardo Silveira Cabral, Tatiana Shavrina, Jakob Foerster, Yoram Bachrach, William Yang Wang, and Roberta Raileanu. 2025. MLGym: A New Framework and Benchmark for Advancing AI Research Agents. arXiv:2502.14499. arXiv:2502.14499 https://arxiv.org/abs/2502.14499

[57] Minh-Duc Nguyen, Hai-Dang Kieu, and Dung D. Le. 2026. AMEM4Rec: Leveraging Cross-User Similarity for Memory Evolution in Agentic LLM Recommenders. arXiv:2602.08837. arXiv:2602.08837 https://arxiv.org/abs/2602.08837

[58] Jinming Nian, Jyun-Yu Jiang, Henry Tsai, Yi Fang, and Zhirong Liang. 2026. Evolving Semantic Indexes for Generative Retrieval. arXiv:2610.28568. arXiv:2610.28568 https://arxiv.org/abs/2610.28568

[59] Jinming Nian, Fangchen Li, Dae Hoon Park, and Yi Fang. 2026. RankEvolve: Automating the Discovery of Retrieval Algorithms via LLM-Driven Evolution. arXiv:2602.16932. arXiv:2602.16932 https://arxiv.org/abs/2602.16932

[60] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131. arXiv:2506.13131 https://arxiv.org/abs/2506.13131

[61] OpenEvolve contributors. 2025. OpenEvolve: An Open-Source Implementation of AlphaEvolve. https://github.com/algorithmicsuperintelligence/openevolve.

[62] Enrico Palumbo, Alexandre Tamborrino, Victor Ode, Ben Lacker, Adrià Casas Escoda, Jeremy Hopple, Marcus Better, James Leoni, Hugo Galvão, Hugues Bouchard, Mounia Lalmas, José Luis Redondo García, Abenezer Abebe, Ann Clifton, Anton Blomberg, Henrik Lindström, Dani Doro, and Christine Doig Cardet. 2026. Bootstrapping Conversational Recommendation Agents At Spotify: Synthetic Data Generation and Self-Improvement Loops. arXiv:2609.30297. arXiv:2609.30297 https://arxiv.org/abs/2609.30297

[63] Weidi Pan, He Ma, Shuhao Ye, Palaksh Rungta, David McPeek, Junyi Jiao, Arnab Bhadury, Mingyan Gao, and Onkar Dalal. 2026. RecEvolve: A Knowledge-Driven Autonomous Agent System for Recommender Systems. arXiv:2609.01622. arXiv:2609.01622 https://arxiv.org/abs/2609.01622

[64] Rayhan Patel and Shabaz Patel. 2026. EvoRank: LLM-Guided Evolution of Multi-Objective Learning-to-Rank Pipelines. arXiv:2609.22196. arXiv:2609.22196 https://arxiv.org/abs/2609.22196

[65] Bo Peng, Lizhu Zhang, Yuhang Zhou, Mingyi Wang, Yifan Wu, Serena Li, Xiangjun Fan, and Zhuokai Zhao. 2026. Discovered, Not Designed: Population Evolution for Collaborative and Compute-Intensive Model Discovery. arXiv:2610.05950. arXiv:2610.05950 https://arxiv.org/abs/2610.05950

[66] Qiyao Peng, Hongtao Liu, Hua Huang, Qing Yang, and Minglai Shao. 2025. A Survey on LLM-powered Agents for Recommender Systems. arXiv:2502.10050. arXiv:2502.10050 https://arxiv.org/abs/2502.10050

[67] Aleksandr Petrov and Craig Macdonald. 2022. A Systematic Review and Replicability Study of BERT4Rec for Sequential Recommendation. arXiv:2207.07483. arXiv:2207.07483 https://arxiv.org/abs/2207.07483

[68] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. arXiv:2305.05065. arXiv:2305.05065 https://arxiv.org/abs/2305.05065

[69] recforge contributors. 2026. recforge: One Protocol to Benchmark and Evolve Recommenders. https://github.com/shangliy/recforge.

[70] Steffen Rendle, Walid Krichene, Li Zhang, and John Anderson. 2020. Neural Collaborative Filtering vs. Matrix Factorization Revisited. arXiv:2005.09683. arXiv:2005.09683 https://arxiv.org/abs/2005.09683

[71] Rebecca Roelofs, Vaishaal Shankar, Benjamin Recht, Sara Fridovich-Keil, Moritz Hardt, John Miller, and Ludwig Schmidt. 2019. A Meta-Analysis of Overfitting in Machine Learning. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*. https://papers.nips.cc/paper/9117-a-meta-analysis-of-overfitting-in-machine-learning

[72] Bernardino Romera-Paredes et al. 2023. Mathematical Discoveries from Program Search with Large Language Models. *Nature* (2023). https://github.com/google-deepmind/funsearch

[73] Xiang Shen, Yuhang Zhou, Yifan Wu, Zhuokai Zhao, Siyu Lin, Lei Huang, Qianqian Zhong, Lizhu Zhang, Benyu Zhang, Xiangjun Fan, and Hong Yan. 2026. Agentic Recommender System with Hierarchical Belief-State Memory. arXiv:2605.14401. arXiv:2605.14401 https://arxiv.org/abs/2605.14401

[74] Qingquan Song, Dehua Cheng, Hanning Zhou, Jiyan Yang, Yuandong Tian, and Xia Hu. 2020. Towards Automated Neural Interaction Discovery for Click-Through Rate Prediction. arXiv:2007.06434. arXiv:2007.06434 https://arxiv.org/abs/2007.06434

[75] Zhu Sun, Hui Fang, Jie Yang, Xinghua Qu, Hongyang Liu, Di Yu, Yew-Soon Ong, and Jie Zhang. 2022. DaisyRec 2.0: Benchmarking Recommendation for Rigorous Evaluation. arXiv:2206.10848. arXiv:2206.10848 https://arxiv.org/abs/2206.10848

[76] Zhen Tao, Riwei Lai, Chenyun Yu, Weixin Chen, Li Chen, Beibei Kong, Lei Cheng, Chengxiang Zhuo, Zang Li, and Qingqiang Sun. 2026. SAGER: Self-Evolving User Policy Skills for Recommendation Agent. arXiv:2604.14972. arXiv:2604.14972 https://arxiv.org/abs/2604.14972

[77] Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Rishi Hazra, Nicolas Baldwin, Alexis Audran-Reiss, Michael Kuchnik, Despoina Magka, Minqi Jiang, Alisia Maria Lupidi, Andrei Lupu, Roberta Raileanu, Kelvin Niu, Tatiana Shavrina, Jean-Christophe Gagnon-Audet, Michael Shvartsman, Shagun Sodhani, Alexander H. Miller, Abhishek Charnalia, Derek Dunfield, Carole-Jean Wu, Pontus Stenetorp, Nicola Cancedda, Jakob Nicolaus Foerster, and Yoram Bachrach. 2025. AI Research Agents for Machine Learning: Search, Exploration, and Generalization in MLE-bench. arXiv:2507.02554. arXiv:2507.02554 https://arxiv.org/abs/2507.02554

[78] Haochen Wang, Yi Wu, Daryl Chang, Li Wei, and Lukasz Heldt. 2026. Self-Evolving Recommendation System: End-To-End Autonomous Model Optimization With LLM Agents. arXiv:2602.10226. arXiv:2602.10226 https://arxiv.org/abs/2602.10226

[79] Lei Wang, Jingsen Zhang, Hao Yang, Zhiyuan Chen, Jiakai Tang, Zeyu Zhang, Xu Chen, Yankai Lin, Ruihua Song, Wayne Xin Zhao, Jun Xu, Zhicheng Dou, Jun Wang, and Ji-Rong Wen. 2023. User Behavior Simulation with Large Language Model based Agents. arXiv:2306.02552. arXiv:2306.02552 https://arxiv.org/abs/2306.02552

[80] Zongwei Wang, Min Gao, Hongzhi Yin, Junliang Yu, Tong Chen, Quoc Viet Hung Nguyen, Shazia Sadiq, and Tianrui Li. 2026. Self-Distilled Reinforcement Learning for Co-Evolving Agentic Recommender Systems. arXiv:2604.10029. arXiv:2604.10029 https://arxiv.org/abs/2604.10029

[81] Zhefan Wang, Yuanqing Yu, Wendi Zheng, Weizhi Ma, and Min Zhang. 2024. MACRec: a Multi-Agent Collaboration Framework for Recommendation. arXiv:2402.15235. arXiv:2402.15235 https://arxiv.org/abs/2402.15235

[82] Chaowei Wu, Huazhu Chen, Congde Yuan, Qirui Yang, Guoqing Song, Yue Gao, Li Luo, Frank Youhua Chen, and Mengzhuo Guo. 2026. AgentLTV: An Agent-Based Unified Search-and-Evolution Framework for Automated Lifetime Value Prediction. arXiv:2602.21634. arXiv:2602.21634 https://arxiv.org/abs/2602.21634

[83] Liwei Wu and Cho-Jui Hsieh. 2026. Closing the Auto-Research Loop: An AI Co-Scientist for Production Search Ranking. arXiv:2603.22376. arXiv:2603.22376 https://arxiv.org/abs/2603.22376

[84] Xidong Wu, Yue Zhuan, Ruoqiao Wei, Hangxin Chen, Di Bai, Jintao Liu, Xinyi Wang, Xue Wang, Luoshu Wang, and Xinwu Cheng. 2026. AgenticRecTune: Multi-Agent with Self-Evolving Skillhub for Recommendation System Optimization. arXiv:2604.26969. arXiv:2604.26969 https://arxiv.org/abs/2604.26969

[85] Zhouhang Xie, Bo Peng, Zhankui He, Ziqi Chen, Alice Han, Isabella Ye, Benjamin Coleman, Noveen Sachdeva, Fernando Pereira, Julian McAuley, Wang-Cheng Kang, Derek Zhiyuan Cheng, Beidou Wang, and Randolph Brown. 2026. AgenticTagger: Structured Item Representation for Recommendation with LLM Agents. arXiv:2602.05945. arXiv:2602.05945 https://arxiv.org/abs/2602.05945

[86] Jinfeng Xu, Zheyu Chen, Ziyue Peng, Zheng Lin, Wenhao Yuan, Jian Chen, Shujie Li, and Edith Ngai. 2026. Beyond Successor Accuracy: State Retention for Recursive Self-Improvement in Recommendation. arXiv:2610.07105. arXiv:2610.07105 https://arxiv.org/abs/2610.07105

[87] Ruiyao Xu, Yanfei Chen, Zhongying CuiZhu, Bhavana Dalvi Mishra, Yifei Ming, Han Yu, Rujun Han, Chen-Yu Lee, and Tomas Pfister. 2026. Harness^2: Recursive Agent Harnessing for an Open World. arXiv:2610.14224. arXiv:2610.14224 https://arxiv.org/abs/2610.14224

[88] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. 2025. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. arXiv:2504.08066. arXiv:2504.08066 https://arxiv.org/abs/2504.08066

[89] Hanchen Yang, Kaiwen Yang, Junpeng Zhuang, Yang He, Keting Cen, Bochao Liu, Zhongbo Sun, An Liu, Zhongteng Han, and Chenyi Lei. 2026. SR-Agent: An Experience-Driven Agentic Framework for Post-Ranking Strategy Refinement in E-Commerce Recommendation. arXiv:2607.17719. arXiv:2607.17719 https://arxiv.org/abs/2607.17719

[90] Shuang Yang, Zijie Zhuang, Changxin Lao, Pengbo Xu, Hanwen Xu, Yusheng Huang, Han Gao, Guanchen Wang, Tianbao Ma, Linxun Chen, Peilin Song, Xuming Wang, Chen Li, Fan Wu, Tao Wang, Zibo Zhao, Xiangyu Wu, An Liu, Fei Pan, Peng Jiang, Chen Yang, Zhaojie Liu, and Wenwu Ou. 2026. Advancing Model Research in AgentX: Long-Horizon Autonomy for Industrial Recommender Systems. arXiv:2609.30001. arXiv:2609.30001 https://arxiv.org/abs/2609.30001

[91] Xu Yang, Xiao Yang, Shikai Fang, Yifei Zhang, Jian Wang, Bowen Xian, Qizheng Li, Jingyuan Li, Minrui Xu, Yuante Li, Haoran Pan, Yuge Zhang, Weiqing Liu, Yelong Shen, Weizhu Chen, and Jiang Bian. 2025. R&D-Agent: An LLM-Agent Framework Towards Autonomous Data Science. arXiv:2505.14738. arXiv:2505.14738 https://arxiv.org/abs/2505.14738

[92] Longfei Yun, Yihan Wu, Haoran Liu, Xiaoxuan Liu, Ziyun Xu, Yi Wang, Yang Xia, Pengfei Wang, Mingze Gao, Yunxiang Wang, Changfan Chen, Fei Liu, Akos Lada, Wenjie Fu, Hong Yan, and Junfeng Pan. 2026. Decoding ML Decision: An Agentic Reasoning Framework for Large-Scale Ranking System. arXiv:2602.18640. arXiv:2602.18640 https://arxiv.org/abs/2602.18640

[93] Wenwen Zeng, Jinhui Zhang, Hao Chen, Zhaoyu Hu, Yongqi Liang, Jiajun Chai, Dengcan Liu, Zhenfeng Liu, Shurui Yan, Minglong Xue, Xiaohan Wang, Wei Lin, and Guojun Yin. 2026. RecRM-Bench: Benchmarking Multidimensional Reward Modeling for Agentic Recommender Systems. arXiv:2605.11874. arXiv:2605.11874 https://arxiv.org/abs/2605.11874

[94] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. arXiv:2402.17152. arXiv:2402.17152 https://arxiv.org/abs/2402.17152

[95] An Zhang, Yuxin Chen, Leheng Sheng, Xiang Wang, and Tat-Seng Chua. 2023. On Generative Agents in Recommendation. arXiv:2310.10108. arXiv:2310.10108 https://arxiv.org/abs/2310.10108

[96] Junjie Zhang, Yupeng Hou, Ruobing Xie, Wenqi Sun, Julian McAuley, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. AgentCF: Collaborative Learning with Autonomous Language Agents for Recommender Systems. arXiv:2310.09233. arXiv:2310.09233 https://arxiv.org/abs/2310.09233

[97] Luankang Zhang, Hao Wang, Zhongzhou Liu, Mingjia Yin, Yonghao Huang, Jiaqi Li, Wei Guo, Yong Liu, Huifeng Guo, Defu Lian, and Enhong Chen. 2026. Can Recommender Systems Teach Themselves? A Recursive Self-Improving Framework with Fidelity Control. arXiv:2602.15659. arXiv:2602.15659 https://arxiv.org/abs/2602.15659

[98] Qi Zhang, Yanlin Chen, and Wenchao Xiao. 2026. AutoLR: Automating the Path from Research to Launch Review in Industrial Recommender Systems. arXiv:2609.04871. arXiv:2609.04871 https://arxiv.org/abs/2609.04871

[99] Sheng Zhang, Maolin Wang, Yao Zhao, Chenyi Zhuang, Jinjie Gu, Ruocheng Guo, Xiangyu Zhao, Zijian Zhang, and Hongzhi Yin. 2024. DNS-Rec: Data-aware Neural Architecture Search for Recommender Systems. arXiv:2402.00390. arXiv:2402.00390 https://arxiv.org/abs/2402.00390

[100] Tunhou Zhang, Dehua Cheng, Yuchen He, Zhengxing Chen, Xiaoliang Dai, Liang Xiong, Feng Yan, Hai Li, Yiran Chen, and Wei Wen. 2022. NAS-Rec: Weight Sharing Neural Architecture Search for Recommender Systems. arXiv:2207.07187. arXiv:2207.07187 https://arxiv.org/abs/2207.07187

[101] Yunxiang Zhang, Muhammad Khalifa, Shitanshu Bhushan, Grant D Murphy, Lajanugen Logeswaran, Jaekyeom Kim, Moontae Lee, Honglak Lee, and Lu Wang. 2025. MLRC-Bench: Can Language Agents Solve Machine Learning Research Challenges? arXiv:2504.09702. arXiv:2504.09702 https://arxiv.org/abs/2504.09702

[102] Yu Zhang, Shutong Qiao, Jiaqi Zhang, Tzu-Heng Lin, Chen Gao, and Yong Li. 2025. A Survey of Large Language Model Empowered Agents for Recommendation and Search: Towards Next-Generation Information Retrieval. arXiv:2503.05659. arXiv:2503.05659 https://arxiv.org/abs/2503.05659

[103] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, Yingqian Min, Zhichao

Feng, Xinyan Fan, Xu Chen, Pengfei Wang, Wendi Ji, Yaliang Li, Xiaoling Wang, and Ji-Rong Wen. 2020. RecBole: Towards a Unified, Comprehensive and Efficient Framework for Recommendation Algorithms. arXiv:2011.01731. arXiv:2011.01731 https://arxiv.org/abs/2011.01731

[104] Xiangyu Zhao, Haochen Liu, Hui Liu, Jiliang Tang, Weiwei Guo, Jun Shi, Sida Wang, Huiji Gao, and Bo Long. 2020. Memory-efficient Embedding for Recommendations. arXiv:2006.14827. arXiv:2006.14827 https://arxiv.org/abs/2006.14827

[105] Xiangyu Zhao, Chong Wang, Ming Chen, Xudong Zheng, Xiaobing Liu, and Jiliang Tang. 2020. AutoEmb: Automated Embedding Dimensionality Search in Streaming Recommendations. arXiv:2002.11252. arXiv:2002.11252 https://arxiv.org/abs/2002.11252

[106] Ruiqi Zheng, Liang Qu, Bin Cui, Yuhui Shi, and Hongzhi Yin. 2022. AutoML for Deep Recommender Systems: A Survey. arXiv:2203.13922. arXiv:2203.13922 https://arxiv.org/abs/2203.13922

[107] Guorui Zhou, Honghui Bao, Jiaming Huang, Jiaxin Deng, Jinghao Zhang, Junda She, Kuo Cai, Lejian Ren, Lu Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rongzhou Zhang, Ruiming Tang, Shiyao Wang, Wuchao Li, Xiangyu Wu, Xinchen Luo, Xingmei Wang, Yifei Hu, Yunfan Wu, Zhanyu Liu, Zhiyang Zhang, Zixing Zhang, Bo Chen, Bin Wen, Chaoyi Ma, Chengru Song, Chenglong Chu, Defu Lian, Fan Yang, Feng Jiang, Hongtao Cheng, Huanjie Wang, Kun Gai, Pengfei Zheng, Qiang Wang, Rui Huang, Siyang Mao, Tingting Gao, Wei Yuan, Yan Wang, Yang Zhou, Yi Su, Zexuan Cheng, Zhixin Ling, and Ziming Li. 2025. OpenOneRec Technical Report. arXiv:2512.24762. arXiv:2512.24762 https://arxiv.org/abs/2512.24762

[108] Guorui Zhou, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rui Huang, Shiyao Wang, Weifeng Ding, Wuchao Li, Xinchen Luo, Xingmei Wang, Zexuan Cheng, Zixing Zhang, Bin Zhang, Boxuan Wang, Chaoyi Ma, Chengru Song, Chenhui Wang, Di Wang, Dongxue Meng, Fan Yang, Fangyu Zhang, Feng Jiang, Fuxing Zhang, Gang Wang, Guowang Zhang, Han Li, Hengrui Hu, Hezheng Lin, Hongtao Cheng, Hongyang Cao, Huanjie Wang, Jiaming Huang, Jiapeng Chen, Jiaqiang Liu, Jinghui Jia, Kun Gai, Lantao Hu, Liang Zeng, Liao Yu, Qiang Wang, Qidong Zhou, Shengzhe Wang, Shihui He, Shuang Yang, Shujie Yang, Sui Huang, Tao Wu, Tiantian He, Tingting Gao, Wei Yuan, Xiao Liang, Xiaoxiao Xu, Xugang Liu, Yan Wang, Yi Wang, Yiwu Liu, Yue Song, Yufei Zhang, Yunfan Wu, Yunfeng Zhao, and Zhanyu Liu. 2025. OneRec Technical Report. arXiv:2506.13695. arXiv:2506.13695 https://arxiv.org/abs/2506.13695

[109] Jieming Zhu, Quanyu Dai, Liangcai Su, Rong Ma, Jinyang Liu, Guohao Cai, Xi Xiao, and Rui Zhang. 2022. BARS: Towards Open Benchmarking for Recommender Systems. arXiv:2205.09626. arXiv:2205.09626 https://arxiv.org/abs/2205.09626

[110] Jieming Zhu, Jinyang Liu, Shuai Yang, Qi Zhang, and Xiuqiang He. 2020. Open Benchmarking for Click-Through Rate Prediction. arXiv:2009.05794. arXiv:2009.05794 https://arxiv.org/abs/2009.05794

[111] Zijie Zhuang, Changxin Lao, Pengbo Xu, Hanwen Xu, Ruochen Yang, Yingzhi He, Peng Zhang, Jiangxia Cao, Yusheng Huang, Guohong Mu, Jian Liang, Ruiming Tang, Shuang Yang, Zhaojie Liu, Wenwu Ou, and Kun Gai. 2026. From Trajectories to Evidence: Auditable Experimental Records for Industrial Research Agents. arXiv:2608.05235. arXiv:2608.05235 https://arxiv.org/abs/2608.05235